GPU-Resident CKKS Inference with PyTorch-Compatible Model Conversion*

Ungsik Kim, Ilhwan Ha, and Suwon Lee

Gyeongsang National University, Jinju, South Korea
{blpeng,mtslzx,leesuwon}@gnu.ac.kr

Abstract

We present our system, a CUDA-accelerated Cheon–Kim–Kim–Song (CKKS) homomorphic encryption library for private neural network inference. Our system combines a PyTorch-compatible model conversion API with a modified OpenFHE-based CUDA backend so that encrypted inference remains resident on the GPU. The system uses device-resident encoded-weight reuse and GPU-resident execution to reduce three practical bottlenecks in the evaluated CKKS inference path: repeated plaintext encoding of model weights, host–device synchronization during rescaling, and CPU fallback for plaintext arithmetic. On encrypted multilayer perceptron inference for MNIST and Fashion-MNIST, our system achieves warm-cache forward-pass speedups of 176–252× over single-threaded CPU, 83–109× over 16-threaded CPU, and 179–185× over TenSEAL. For a 256-hidden-unit MLP, warm-cache end-to-end latency is 379 ms, while encrypted accuracy remains within one percentage point of plaintext baselines.

1 Introduction

Cloud machine learning services increasingly process sensitive inputs such as medical records, financial transactions, and biometric measurements. Homomorphic encryption (HE) offers a direct privacy-preserving approach: clients encrypt their inputs, servers evaluate a model over ciphertexts, and only clients decrypt the result. Among HE schemes, CKKS [1] is especially suited to neural network inference because it supports approximate fixed-point arithmetic over packed vectors.

This capability is directly relevant to the evolution of electronic business and commerce information systems. Modern e-commerce platforms increasingly rely on cloud-hosted AI services for fraud detection, payment-risk scoring, personalized recommendation, credit and insurance decisions, biometric authentication, and privacy-sensitive healthcare or financial workflows. These services must extract value from user data while complying with stricter privacy expectations and data-protection requirements. Encrypted inference addresses this tension by allowing a service provider to evaluate a deployed model without observing the user's raw input, which makes it a practical building block for privacy-preserving decision support in data-sensitive e-commerce environments.

The main obstacle is performance. CKKS inference spends most of its time in polynomial arithmetic, rotations, rescaling, and plaintext encoding. GPUs are a natural target for this workload, but accelerating individual primitives is not enough for practical inference. Existing systems often leave parts of the pipeline on the CPU, require manual circuit construction, or repeatedly encode static model weights for every request. These gaps are particularly costly in inference-as-a-service settings, where a fixed model is evaluated many times on different encrypted inputs.

Our system is a GPU-resident CKKS inference system designed around this deployment model. It exposes a PyTorch-compatible API that converts supported trained models into encrypted modules,

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 29, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

then evaluates the resulting computation through a modified OpenFHE-based CUDA backend. The system focuses on the operations that dominate encrypted neural inference: linear layers implemented with Baby-step Giant-step (BSGS) rotations, polynomial activations, ciphertext rescaling, and plaintext arithmetic for weights and biases.

This paper makes three contributions. First, our system integrates the CKKS operations exercised by our benchmarks into a GPU-resident inference path, including ciphertext–plaintext arithmetic, rotations, and rescaling. Second, it provides automatic model conversion from PyTorch modules, reducing the amount of cryptographic expertise needed to deploy private inference. Third, it evaluates the resulting warm-cache service path, showing 176–252× speedup over single-threaded CPU and 179–185× speedup over TenSEAL while preserving classification accuracy within one percentage point of plaintext models.

2 Background and Design

2.1 CKKS and BSGS Linear Layers

CKKS is a leveled HE scheme over the ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$. A plaintext vector is encoded into a polynomial, encrypted into a ciphertext, and then manipulated through homomorphic addition, multiplication, rotation, and rescaling. Rotation is central to neural inference because it moves packed slots inside a ciphertext; it is also expensive because it requires key switching. Rescaling restores the scale after multiplication and consumes one level from the modulus chain.

Our system evaluates linear layers using the BSGS method [2]. Given a $d \times d$ weight matrix W and encrypted input ct, the matrix is decomposed into generalized diagonals p_k. BSGS chooses $d_1 d_2 \geq d$ and computes

$$W \cdot ct = \sum_{j=0}^{d_2-1} \mathrm{Rot}\Big(\sum_{i=0}^{d_1-1} p_{i+jd_1} \cdot \mathrm{Rot}(ct, i), jd_1 \Big). \tag{1}$$

This reduces the number of rotations from d to roughly $2\sqrt{d}$, but it requires each weight diagonal to be encoded as a plaintext polynomial. For static model weights, repeatedly performing this encoding is wasteful and becomes the dominant cold-start cost on GPU.

2.2 System Architecture

Our system is organized as a Python conversion layer, encrypted neural-network modules, and a modified OpenFHE-based CUDA backend. The top layer accepts a trained PyTorch model and recursively replaces supported modules with encrypted counterparts. Linear layers become BSGS-based encrypted linear layers; convolutional layers use an im2col lowering followed by BSGS; ReLU, GELU, and SiLU activations are replaced by Chebyshev polynomial approximations; and batch normalization can be folded into preceding affine layers at conversion time. The encrypted modules call a pybind11 extension backed by a modified OpenFHE GPU fork.

2.3 Threat Model and Security Assumptions

Our system targets an honest-but-curious inference server. The client generates the CKKS secret key locally, encrypts inputs, sends ciphertexts and evaluation keys to the server, and decrypts the returned ciphertext outputs. The server follows the prescribed protocol but may observe all ciphertexts, evaluation keys, and GPU-resident encrypted intermediate buffers generated during execution. Under the standard

CKKS security assumptions, these encrypted values do not reveal the client's plaintext input or input-derived intermediate activations.

Model weights are assumed to be public or available to the service provider, so model confidentiality is not a target of this work. The evaluated computation has a fixed circuit structure for a given model and parameter set, but we do not provide a formal analysis of timing or microarchitectural side channels. Multi-user GPU isolation and defenses against malicious clients are treated as deployment-hardening concerns for production e-commerce services rather than security guarantees of the proposed inference engine.

2.4 GPU-Resident Optimizations

Our system relies on three implementation choices to keep the evaluated warm-cache inference path resident on the GPU.

Device-resident plaintext reuse. For each converted model, our system reuses encoded weight diagonals stored in device memory. Cache keys include the layer, diagonal index, BSGS parameters, ciphertext level, and a weight hash. The first encrypted forward pass populates these entries; later requests reuse the encoded plaintexts directly on the GPU. For the evaluated three-layer MLPs, the number of cached diagonals is 1,040 ($h = 128$), 1,296 ($h = 256$), and 1,808 ($h = 512$), all within the default cache capacity of 2,048 entries. Under the parameters used in this paper, a plaintext entry occupies about 3.2 MB, so the full cache uses about 6.4 GB of GPU memory.

Zero-copy rescaling. Naive BSGS accumulation can allocate and copy ciphertext buffers at each ciphertext–plaintext multiplication and rescale. CUDA allocation introduces implicit synchronization, which serializes what should be a device-resident pipeline. Our system modifies the accumulation path to transfer C++ buffer ownership with move semantics and to reuse GPU buffers whenever possible. This avoids deep-copying approximately 6.4 MB ciphertext objects during accumulation.

GPU-native plaintext arithmetic. The OpenFHE GPU fork used as a base lacked GPU-native plaintext addition and subtraction in the inference path exercised by our system. That limitation forced a CPU round trip for bias addition. Our system adds in-place CUDA kernels for these operations so that linear-layer bias handling does not break GPU residency.

The experimental configuration uses ring dimension $N = 32{,}768$, 16,384 slots, scale $\Delta = 2^{50}$, and a ternary secret distribution. The three-layer MLP uses an 11-level modulus chain with at least 128-bit security under the Homomorphic Encryption Standard [3].

3 Evaluation

3.1 Setup

We evaluate our system on an RTX 5090 GPU with 32 GB GDDR7 memory and an Intel Core Ultra 9 285K CPU running Ubuntu 22.04 and CUDA 12.8. The HE backend is a modified OpenFHE-based CUDA build. The main benchmark is a three-layer MLP, $784 \to h \to h \to 10$, with $h \in \{128, 256, 512\}$, evaluated on MNIST and Fashion-MNIST. Inputs are scaled to $[-1, 1]$, and ReLU is approximated by a degree-4 Chebyshev polynomial. Models are trained with Adam for 20 epochs using learning rate 10^{-3} and batch size 128.

Table 1: Forward-Pass Latency Across Systems in Seconds

System	$h = 128$	$h = 256$	$h = 512$
TenSEAL	16.233	17.920	21.813
Ours-CPU 1T	15.630	20.189	29.712
Ours-CPU 16T	7.388	9.251	12.809
Ours-GPU warm	0.089	0.100	0.118
Speedup vs. CPU 1T	176×	202×	252×
Speedup vs. CPU 16T	83×	93×	109×
Speedup vs. TenSEAL	182×	179×	185×

Table 2: Cold and Warm Latency Breakdown for Ours-GPU, $h = 256$

Component	Cold (ms)	Warm (ms)
Client encrypt	267	267
Server forward pass	3029	101
Client decrypt	11	11
End-to-end	3307	379

We report forward-pass latency for encrypted model evaluation and, where noted, end-to-end latency including client encryption and decryption. GPU forward-pass measurements use the warm cache unless stated otherwise. TenSEAL v0.3.16 with Microsoft SEAL is used as a CPU-based Python encrypted tensor baseline under matched CKKS parameters.

3.2 CPU and Framework Comparison

Table 1 compares our system with TenSEAL and CPU OpenFHE baselines. Our system reaches 89–118 ms warm-cache forward-pass latency across the three MLP widths. This corresponds to 176–252× speedup over single-threaded CPU and 179–185× over TenSEAL; even against 16 CPU threads, the speedup remains 83–109×.

The GPU latency increases by only 33% when h grows from 128 to 512, while single-threaded CPU latency increases by 90%. CPU parallelism helps but saturates well above the GPU latency, indicating that the workload benefits from GPU memory bandwidth and massive polynomial-level parallelism.

3.3 Cold and Warm Inference

Device-resident plaintext reuse is the largest measurable contributor to warm-cache performance. Disabling it increases GPU forward-pass latency by 14.6–15.4× because every request must re-encode all BSGS weight diagonals. Table 2 shows the end-to-end effect for the $h = 256$ MLP. The first request takes 3.307 s because cache population dominates the server forward pass. Once populated, the same model reaches 379 ms end-to-end latency.

3.4 Accuracy and Generalization

Encrypted accuracy remains close to plaintext accuracy. On MNIST, our system achieves 98.07%, 98.09%, and 98.15% accuracy for $h = 128, 256, 512$, respectively, within 0.06 percentage points of the

corresponding plaintext models. On Fashion-MNIST, encrypted accuracy is 88.92–89.66%, within one percentage point of plaintext accuracy for all widths. We also evaluated a small encrypted convolutional model, Conv2d$(1,8,k=5,s=4)$ plus a fully connected layer, which reaches 95.9% encrypted accuracy on 1,000 MNIST samples with 0.227 s warm-cache latency.

3.5 Scope and Limitations

The current evaluation focuses on latency-sensitive single-query inference for small MLP and CNN models. This is the regime where repeated plaintext encoding and CPU fallback are especially visible, and where a warm-cache service model is most natural. Larger networks introduce additional constraints: rotation keys grow with the set of slot permutations needed by packed convolutions, cached plaintexts compete with ciphertexts and evaluation keys for GPU memory, and deeper polynomial circuits require longer modulus chains or bootstrapping.

The reported speedups should therefore be interpreted as end-to-end results for our evaluated inference path, not as a general comparison against all GPU CKKS libraries. TenSEAL is used as a CPU-based Python tensor baseline, while recent GPU CKKS libraries such as HEonGPU, Cheddar, and FIDESlib focus on lower-level primitive coverage and bootstrapping. A cache-matched CPU baseline and direct comparisons with standalone GPU CKKS backends would further clarify how much of the gain comes from GPU execution, cache reuse, and framework-level integration.

3.6 Deployment Implications

The warm-cache measurements are most relevant to persistent inference services where a provider loads one encrypted model, populates the encoded-weight cache on the first request, and then serves many encrypted inputs. In this setting, the cold-start cost is amortized, and the main latency floor becomes client encryption, GPU-resident model evaluation, and client decryption. For the $h = 256$ MLP, the server-side warm forward pass is 101 ms, while the full encrypt–evaluate–decrypt path is 379 ms. The first request is substantially slower because it must populate the encoded-weight cache; in our $h = 256$ measurement, this cold path takes 3.307 s end-to-end. Thus, the reported warm latency should be interpreted as the steady-state behavior of a long-running service rather than the cost of loading a new model or serving a rarely used model for the first time. Future batching or client-side encryption optimizations would therefore affect a different part of the pipeline than the GPU cache.

The same design exposes a memory tradeoff. Pinning encoded diagonals makes warm inference fast, but the cache shares GPU memory with ciphertexts and evaluation keys. The evaluated MLPs fit within the default 2,048-entry cache, but larger packed CNNs and multi-model services may face higher pressure from cached plaintexts, rotation keys, ciphertext working buffers, and larger model weights. In such deployments, cache capacity determines a tradeoff between steady-state latency and available GPU memory. Cache admission policies that account for layer reuse, diagonal sparsity, and available device memory are therefore important for scaling the approach beyond the evaluated models. This is why we view cache management and rotation-key planning as system-level problems rather than only low-level kernel optimizations.

3.7 Measurement Interpretation

The latency numbers in this paper should be read as service-path measurements rather than isolated primitive benchmarks. A primitive-level comparison can show whether a particular NTT, key-switch, or rescale kernel is fast, but it does not capture whether an encrypted model repeatedly crosses the host–device boundary or reconstructs static plaintexts for every query. Our evaluation therefore separates cold and warm behavior and reports both server forward time and encrypt–evaluate–decrypt time. This

distinction is important for encrypted inference services because the client-visible latency includes operations that are outside the GPU evaluator, while the server-side latency is the component most directly affected by GPU residency.

The comparison with TenSEAL has the same scope. TenSEAL is a mature Python interface for encrypted tensor operations and is a natural framework-level baseline for users, but it is not a GPU CKKS backend. Conversely, GPU CKKS libraries such as HEonGPU, Cheddar, and FIDESlib provide broader low-level acceleration targets but do not directly evaluate the same PyTorch model conversion workflow. The intended reading is therefore that the proposed integration substantially reduces latency relative to CPU-oriented framework baselines, while direct GPU-library comparisons remain future work.

4 Related Work

CryptoNets introduced encrypted neural inference [4]. Hybrid systems such as GAZELLE, Delphi, and Cheetah combine HE with secure multi-party computation but require interaction [5, 6, 7]. CHET, EVA, nGraph-HE, TenSEAL, and HELayers improve HE programmability and compiler support [8, 9, 10, 11, 12]. Hardware-oriented and GPU library work accelerates HE primitives, encrypted CNN inference, and bootstrapping [13, 14, 15, 16, 17, 18]. Our system is complementary: it focuses on a pure-HE, PyTorch-facing inference path whose warm-cache evaluation remains GPU-resident for the evaluated models.

5 Conclusion

Our system is a GPU-resident CKKS inference system with PyTorch model conversion. Device-resident plaintext reuse, zero-copy rescaling, and GPU-native plaintext arithmetic remove CPU fallbacks in the evaluated path and reduce repeated encoding costs for static model weights. Our system achieves 176–252$\times$ speedup over single-threaded CPU and 179–185$\times$ over TenSEAL, with sub-400 ms warm-cache end-to-end latency for $h = 256$. Future work includes studying which optimization techniques from conventional deep learning, such as operator fusion, architecture compression, memory scheduling, and larger CNN or ResNet-style designs, remain effective under CKKS constraints where rotations, rescaling, ciphertext levels, and VRAM usage dominate the cost model. A related direction is VRAM-aware system design, including cache admission, rotation-key layout, and encrypted-buffer reuse for larger models and multi-model inference services.

Acknowledgments

This research was supported by the Regional Innovation System & Education (RISE) program through the RISE Center, Gyeongsangnam-do, funded by the Ministry of Education (MOE) and the Gyeongsangnam-do Provincial Government, Republic of Korea. (2025-RISE-16-001)

References

[1] Cheon, J. H., Kim, A., Kim, M., & Song, Y. (2017). Homomorphic encryption for arithmetic of approximate numbers. In *Lecture Notes in Computer Science* (pp. 409–437). Springer International Publishing. https://doi.org/10.1007/978-3-319-70694-8_15

[2] Halevi, S., & Shoup, V. (2018). Faster homomorphic linear transformations in HElib. In *Lecture Notes in Computer Science* (pp. 93–120). Springer International Publishing. https://doi.org/10.1007/978-3-319-96884-1_4

[3] Albrecht, M., et al. (2018). *Homomorphic encryption security standard*. HomomorphicEncryption.org.

[4] Dowlin, N., Gilad-Bachrach, R., Laine, K., Lauter, K., Naehrig, M., & Wernsing, J. (2016). CryptoNets: Applying neural networks to encrypted data with high throughput and accuracy. In *Proceedings of the International Conference on Machine Learning* (pp. 201–210).

[5] Juvekar, C., Vaikuntanathan, V., & Chandrakasan, A. (2018). GAZELLE: A low latency framework for secure neural network inference. In *Proceedings of the USENIX Security Symposium* (pp. 1651–1669).

[6] Mishra, P., Lehmkuhl, R., Srinivasan, A., Zheng, W., & Popa, R. A. (2020). Delphi: A cryptographic inference service for neural networks. In *Proceedings of the USENIX Security Symposium* (pp. 2505–2522).

[7] Huang, Z., Lu, W.-J., Hong, C., & Ding, J. (2022). Cheetah: Lean and fast secure two-party deep neural network inference. In *Proceedings of the USENIX Security Symposium* (pp. 809–826).

[8] Dathathri, R., Saarikivi, O., Chen, H., Laine, K., Lauter, K., Maleki, S., Musuvathi, M., & Mytkowicz, T. (2019). CHET: An optimizing compiler for fully homomorphic neural-network inferencing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (pp. 142–156). ACM. https://doi.org/10.1145/3314221.3314628

[9] Dathathri, R., Kostova, B., Saarikivi, O., Dai, W., Laine, K., & Musuvathi, M. (2020). EVA: An encrypted vector arithmetic language and compiler for efficient homomorphic computation. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (pp. 546–561). ACM. https://doi.org/10.1145/3385412.3386023

[10] Boemer, F., Lao, Y., Cammarota, R., & Wierzynski, C. (2019). nGraph-HE: A graph compiler for deep learning on homomorphically encrypted data. In *Proceedings of the 16th ACM International Conference on Computing Frontiers* (pp. 3–13). ACM. https://doi.org/10.1145/3310273.3323047

[11] Benaissa, A., Retiat, B., Cebere, B., & Belfedhal, A. E. (2021). TenSEAL: A library for encrypted tensor operations using homomorphic encryption. *arXiv*. https://doi.org/10.48550/arXiv.2104.03152

[12] Aharoni, E., Adir, A., Baruch, M., Drucker, N., Ezov, G., Farkash, A., Greenberg, L., Masalha, R., Moshkowich, G., Murik, D., Shaul, H., & Soceanu, O. (2023). HeLayers: A tile tensors framework for large neural networks on encrypted data. *Proceedings on Privacy Enhancing Technologies, 2023*(1), 325–342. https://doi.org/10.56553/popets-2023-0020

[13] Jung, W., Kim, S., Ahn, J. H., Cheon, J. H., & Lee, Y. (2021). Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with GPUs. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 114–148. https://doi.org/10.46586/tches.v2021.i4.114-148

[14] Kim, S., Kim, J., Kim, M. J., Jung, W., Kim, J., Rhu, M., & Ahn, J. H. (2022). BTS: An accelerator for bootstrappable fully homomorphic encryption. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (pp. 711–725). ACM. https://doi.org/10.1145/3470496.3527415

[15] Al Badawi, A., Chao, J., Lin, J., Mun, C. F., Sim, J. J., Tan, B. H. M., Nan, X., Aung, K. M. M., & Chandrasekhar, V. R. (2021). Towards the AlexNet moment for homomorphic encryption: HCNN, the first homomorphic CNN on encrypted data with GPUs. *IEEE Transactions on Emerging Topics in Computing, 9*(3), 1330–1343. https://doi.org/10.1109/TETC.2020.3014636

[16] Özcan, A. Ş., & Savaş, E. (2024). *HEonGPU: A GPU-based fully homomorphic encryption library 1.0*. Cryptology ePrint Archive, Paper 2024/1543. https://eprint.iacr.org/2024/1543

[17] Kim, J., et al. (2024). *Cheddar: A swift fully homomorphic encryption library for CUDA GPUs*. *arXiv*. https://arxiv.org/abs/2407.13055

[18] Agulló-Domingo, C., Vera-López, Ó., Guzelhan, S., Daksha, L., El Jerari, A., Shivdikar, K., Agrawal, R., Kaeli, D., Joshi, A., & Abellán, J. L. (2025). *FIDESlib: A fully-fledged open-source FHE library for efficient CKKS on GPUs*. *arXiv*. https://arxiv.org/abs/2507.04775